



Computer Science Competition State 2021 Programming Problem Set

I. General Notes

1. Do the problems in any order you like. They do not have to be done in order from 1 to 12.
2. All problems have a value of 60 points.
3. There is no extraneous input. All input is exactly as specified in the problem. Unless specified by the problem, integer inputs will not have leading zeros. Unless otherwise specified, your program should read to the end of file.
4. Your program should not print extraneous output. Follow the form exactly as given in the problem.
5. A penalty of 5 points will be assessed each time that an incorrect solution is submitted. This penalty will only be assessed if a solution is ultimately judged as correct.

II. Names of Problems

Number	Name
Problem 1	Adriana
Problem 2	Bruno
Problem 3	Delfina
Problem 4	Eszter
Problem 5	Hachiro
Problem 6	Irene
Problem 7	Joseph
Problem 8	Khushi
Problem 9	Maite
Problem 10	Mayank
Problem 11	Rashi
Problem 12	Sergio

1. Adriana

Program Name: Adriana.java

Input File: none

Adriana is learning about Gray codes. An n -bit Gray code is an ordering of the first 2^n binary numbers such that any consecutive numbers in the order differ in exactly one bit. For example, a valid 2-bit Gray code is:

```
00
01
11
10
```

One common algorithm to generate an n -bit Gray code is to first generate two copies of an $(n - 1)$ bit Gray code. Prepend a '0' bit to every number in the first copy, prepend a '1' to every number in the second copy, and reverse the order of the second copy. Finally, concatenate the first and second copies to get the n -bit Gray code.

For example, to generate a 3-bit Gray code using the above 2-bit Gray code, we first take our two copies as shown in left panel and prepend the appropriate bits to each copy as shown on right:

00	00	000	100
01	01	001	101
11	11	011	111
10	10	010	110

Finally, reverse the second copy and concatenate:

```
000
001
011
010
110
111
101
100
```

Adriana thinks she has this procedure down, but wants to compare with another person to make sure her 4-bit Gray code is correct. Help her out by writing a program that outputs the 4-bit Gray code.

Input: There is no input for this problem.

Output: Output the 4-bit Gray code that results from applying the above algorithm to the given 3-bit Gray code.

Sample input and output continued on next page ...

Adriana continued ...

Sample output:

```
0000
0001
0011
0010
0110
0111
0101
0100
1100
1101
1111
1110
1010
1011
1001
1000
```

Sample explanation: Note that for every pair of consecutive entries in the list, they differ by exactly one bit. Furthermore, using this method, the first and last entries also differ by exactly one bit!

2. Bruno

Program Name: Bruno.java

Input File: bruno.dat

Bruno once witnessed a flock of snow geese and read an article about them he found on the Web. He decided on a whim to analyze the article to see how many words were in the article. Help Bruno do this by writing a program that reads a data file and counts the number of words in the paragraph in the data file.

The annual migration of Snow Geese is one of the greatest wildlife spectacles in North America. Snow geese spend winters in the coastal marshes and interior wetlands of the United States and find summer breeding grounds in the tundra of the Alaskan and Canadian Arctic, stopping daily to feed on roots and unharvested grain. Snow Geese travel and forage in noisy flocks often numbering in the tens of thousands and follow a cycle that requires them to fly thousands of miles each year. The flock is able to change direction rapidly because birds can mimic the maneuvers of distant flock members and do not wait for cues from adjacent birds.

Input: A data file containing a single paragraph of several sentences on several lines. All words will be separated by at least one space. There will be commas and periods immediately following some words, but no other punctuation will be included. Length of paragraph will not exceed 500 words.

Output: A single integer representing the number of words in the paragraph.

Sample input: (the paragraph shown above)

Sample output:

110

3. Delfina

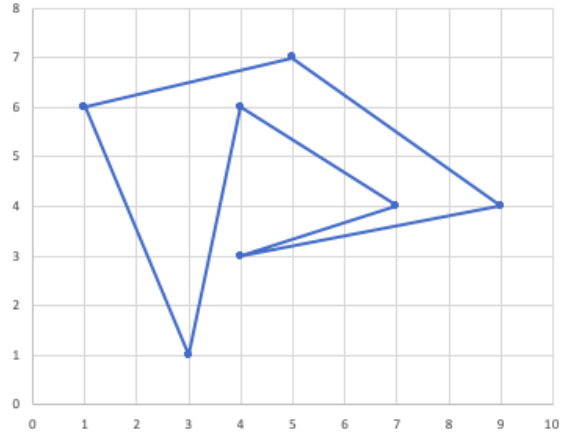
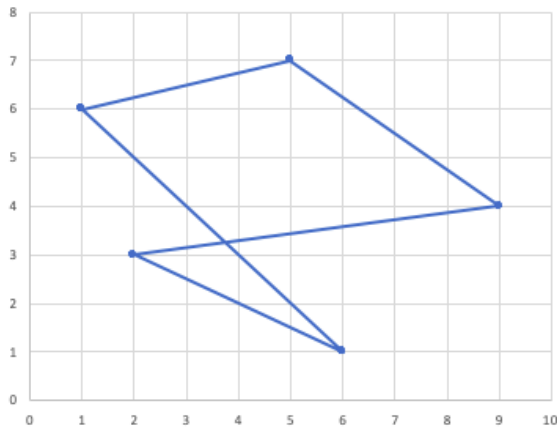
Program Name: Delfina.java

Input File: delfina.dat

A *polygon* P determined by points $p_1, p_2, \dots, p_n$ is a closed chain of line segments (called edges) $p_1, p_2, \dots, p_n, p_1$ in a single plane. Polygon P is *simple*, if no two edges have any points in common, with the obvious exception of two consecutive segments having one common point (called a vertex).

Any polygon that is not simple is called *self intersecting*. In the two example figures below, the left polygon (Sample Input Test Case #1) is self intersecting, the right polygon (Sample Input Test Case #2) is simple.

Delfina is asking you to help her determine if a given polygon is simple or self intersecting. Can you help her with this?



Input: Input starts with a line containing an integer N ($1 \leq N \leq 20$), the number of test cases. Each test case will begin with an integer M ($1 \leq M \leq 50$) where M is the number of vertices for the given polygon. The following M lines will correspond with an X and Y component for the given vertex. X and Y will be separated by a single space. Both X and Y will be integers in range of $[0, 15]$. An edge exists between the current vertex and the vertex prior. For the first vertex of a test case, an edge exists between the first vertex and the last vertex. All vertices in the input are distinct.

Output: For each test case, output “Simple” if the given polygon has no two points in common (except for vertices) and no intersecting edges, or output “Self intersecting” if the given polygon has at least one pair of edges that intersect.

Sample input and output continued on next page ...

Delfina continued ...

Sample input:

```
6
5
1 6
5 7
9 4
2 3
6 1
7
1 6
5 7
9 4
4 3
7 4
4 6
3 1
4
4 4
5 2
2 3
3 1
3
4 3
3 1
2 2
4
4 3
3 1
2 2
3 4
4
1 1
3 2
4 1
2 2
```

Sample output:

```
Self intersecting
Simple
Self intersecting
Simple
Simple
Self intersecting
```

4. Eszter

Program Name: Eszter.java

Input File: eszter.dat

Eszter's dad owns a wood mill. After taking the logs from the lumberjacks and cutting the needed dimensional lumber requested by his clients, Eszter's dad has remaining pieces of wood left over that can still be sold for profit. The problem he faces is how to maximize the profit with the left-over pieces. He knows how much money can be made by cutting an N foot leftover piece into 1, 2, 3, ..., N-1, and N foot pieces, but has no idea the maximum value this N foot piece is worth. For example, assume Eszter's dad has an 8-foot piece of leftover oak left, and the below table shows the profit that can be made from cutting the oak into 1, 2, ..., 7, and 8-foot pieces.

Length	1	2	3	4	5	6	7	8
Price	\$1.00	\$5.00	\$8.00	\$9.00	\$10.00	\$17.00	\$17.00	\$20.00

The maximum value of this 8-foot piece of oak wood would be \$22.00 since the 8-foot piece could be cut into a 2-foot section as well as a 6-foot section (\$5.00+\$17.00=\$22.00).

However, Eszter's dad is not limited by a minimum or maximum number of cuts. For example, given the following information for an 8-foot piece of pine, Eszter's dad could maximize his profit by cutting the piece of wood into 8 pieces each 1 foot long. (8*\$3.00=\$24.00)

Length	1	2	3	4	5	6	7	8
Price	\$3.00	\$5.00	\$8.00	\$9.00	\$10.00	\$17.00	\$17.00	\$20.00

Eszter's dad has asked Eszter to write a program that can tell him the maximum worth of a piece of N-foot long wood given the prices for pieces 1, 2, 3, ..., N-1, and N. Can you help Eszter with this?

Input: Input starts with a line containing an integer Z ($1 \leq Z \leq 20$), the number of test cases. Each test case is made up of a set of prices, comma separated, in curly braces corresponding to the price for pieces of length 1, 2, 3, ..., N-1, N. N will be in range [1,20]. Prices will be given as floating-point numbers, formatted to two decimal places. Prices will be in range [0.00,100.00]. NOTE: the sample input for Test Case #4 below is word-wrapped for formatting, however, this is all still one line of input.

Output: For each test case, output, "Maximum profit is \$VALUE", where value is the maximum profit that could be made formatted to two decimal places.

Sample input:

```
4
{1.00,5.00,8.00,9.00,10.00,17.00,17.00,20.00}
{3.00,5.00,8.00,9.00,10.00,17.00,17.00,20.00}
{1.50,2.25,3.10,4.75,5.00}
{18.74,20.99,24.11,28.78,28.79,35.11,41.56,44.04,52.44,57.54,59.98,60.11,
70.89,71.61,75.92,84.30,88.57,89.16,95.11,95.87}
```

Sample output:

```
Maximum profit is $22.00
Maximum profit is $24.00
Maximum profit is $7.50
Maximum profit is $374.80
```

5. Hachiro

Program Name: Hachiro.java

Input File: hachiro.dat

Hachiro has a web site where he publishes the table (standings) for his high school's district soccer teams. Standings are based on points in soccer where, if there is a winner in a match, the winner receives 3 points and the loser none. If there is a tie, both teams receive 1 point. All of the scores for every match played within the district are stored in a file. Hachiro needs a program to print the table for display on his web site. Write that program for Hachiro.

Input: A single value N ($N \leq 20$) representing the number of matches that have been played. The next N lines will each contain the home team's name (all one word mascot names), a space, the home team's score, a space, the visiting team's name, a space, and the visiting team's score for each match.

Output: A line with the heading "6A District 33 Soccer Table". On the next line there should be six column headings as follows: "Team", "W", "T", "L", "P", and "GD". On the remaining lines print the table with columns for the team name, wins, ties, losses, points and goal differential. Teams should be listed in descending order by points. In the event of a tie in the standings, break the tie based on the team with the greatest goal differential i.e. total goals scored by the team minus the total goals scored against the team.

Format the column headings and data as follows:

Team name left justified in 8 spaces.

Wins, ties, losses, and points right justified in 3 spaces.

Goal differential right justified in 4 spaces.

Sample input:

```
15
Lions 3 Tigers 2
Tigers 4 Eagles 1
Bulldogs 1 Tigers 1
Tigers 2 Panthers 2
Wildcats 0 Tigers 4
Lions 5 Eagles 1
Bulldogs 0 Lions 0
Lions 1 Panthers 0
Wildcats 2 Lions 1
Eagles 1 Bulldogs 3
Panthers 3 Eagles 2
Eagles 4 Wildcats 4
Panthers 1 Bulldogs 1
Bulldogs 5 Wildcats 4
Wildcats 2 Panthers 1
```

Sample output:

```
6A District 33 Soccer Table
Team      W  T  L  P  GD
Lions      3  1  1 10  5
Bulldogs   2  3  0  9  3
Tigers     2  2  1  8  6
Wildcats   2  1  2  7 -3
Panthers   1  2  2  5 -1
Eagles     0  1  4  1 -10
```


6. Irene

Program Name: Irene.java

Input File: irene.dat

To fundraise and increase brand recognition, the United Imaging Laboratory is selling customizable displays. A display is a table of R rows and C columns, where each cell in the table can be turned on or off by inputting a U, I or L pattern. To customize a table, a user can select one or more of the three following patterns and place them somewhere on the grid such that no two patterns overlap. Two patterns overlap if they share at least one cell. The patterns cannot be rotated or modified before placing.

```
#.#      #      #..
#.#      #      #..
###      #      ###
```

The #s represent cells that are turned on, while the dots represent blank cells. Patterns can be placed in blank cells adjacent to other patterns. For example, here's a U sitting adjacent to an L.

```
.U.U
LU.U
LUUU
LLL.
```

Given a grid where R = 3 and C = 9, two possible ways of many to fill in the grid could be:

```
#.#.#.#..      #...#.#.#
#.#.#.#..      #...#.#.#
###.#.###      ###.#.###
```

where dots represent blank cells.

Unfortunately, on some displays, some of the cells are defective and cannot be turned on. For example, if an R = 3, C = 9 grid has the following defective cells, marked by asterisks:

```
.*. . *. *.
..* . . **
* . . ** . . *
```

then it is impossible to place any U, I, or L patterns.

Given a display, how many distinct ways are there to place one or more patterns such that no two patterns overlap? Two ways are different if there is a pattern placed differently between the two grids or on one grid but not the other.

Input: Input starts with a line containing an integer T ($1 \leq T \leq 20$), the number of test cases. Each test case begins with two integers R and C ($1 \leq R, C \leq 10$), the number of rows and number of columns. The following R rows each have C characters. A '.' character means the cell is empty, and an '*' character means the cell is defective.

Output: For each test case, output the number of different ways to place one or more patterns on the grid.

Sample input and output continued on next page ...

Irene continued ...

Sample input:

```
4
3 9
.....
.....
.....
3 9
.*...*.*.
.*...**.*
*...**...*
3 3
...
...
...
6 4
....
.*.
*...
....
....
....
```

Sample output:

```
1735
0
9
404
```

7. Joseph

Program Name: Joseph.java

Input File: joseph.dat

Joseph is developing a dating app. He has created a questionnaire that asks a variety of questions about a person's likes, dislikes, personality traits and other characteristics about that person that he then compiles into a personality score. The scores are always between 1 and 100 inclusive. A score of 1 or 100 is neither bad nor good. The scores simply indicate that person's personality traits according to Joseph. Joseph's theory of dating is that people with the most similar scores on his questionnaire are most likely to be a good match. He also believes that two people must score within 10 points of one another to be matched at all. The problem is that Joseph is a much better match maker than he is a programmer. He needs for your team to write a program that will take two separate lists of potential daters and find all the couples that score within 10 points of one another and then list the best matches in order from the very best match (smallest difference in scores) to worst match (greatest difference in scores). Only potential daters may see the names of their matches so all of the people will be listed using just their first, middle and last initials.

Input: There will always be exactly four lines of data. The first line will be a list of daters initials, each separated by one space, that represent a group of potential daters. The second line will have a list of scores on the questionnaire that corresponds to each of the potential daters in the previous line. There will always be the same number of daters as there are scores in the first two lines. The third line will contain a list of daters initials, each separated by one space, that represent a second group of daters that are to be matched with the first group. The group sizes do not necessarily match. The fourth and final line will contain the corresponding scores from the questionnaire for the second group. There will always be the same number of daters as there are scores in the third and fourth lines. There will never be more than 20 daters in either group.

Output: A line for each match that lists the initials for each potential couple and the positive difference between their scores on the questionnaire with one space between the initials and one space between the initials and the difference. The list should be shown in ascending order based on the difference then alphabetically by the dater from the first list and finally by the dater from the second list. Do not include any matches with a difference greater than ten.

Sample input:

```
TRQ CTU QOI JYN PEV NIL BZZ VAS EBC
51 1 50 100 75 12 62 43 38
ALL STR DJP FBO YWA RIK OOO WFS ZZT LOK KAR
38 19 60 70 79 9 51 88 60 50 51
```

Sample output:

```
EBC ALL 0
QOI LOK 0
TRQ KAR 0
TRQ OOO 0
QOI KAR 1
QOI OOO 1
TRQ LOK 1
BZZ DJP 2
BZZ ZZT 2
NIL RIK 3
PEV YWA 4
PEV FBO 5
VAS ALL 5
NIL STR 7
VAS LOK 7
BZZ FBO 8
CTU RIK 8
VAS KAR 8
VAS OOO 8
TRQ DJP 9
TRQ ZZT 9
QOI DJP 10
QOI ZZT 10
```

8. Khushi

Program Name: Khushi.java

Input File: khushi.dat

Khushi is writing code for a cash register company. For now she is working on the code to handle cash transactions and more specifically the class to calculate the change owed to a customer. A customer's change should always be given using the fewest bills and coins as possible. The largest bill that will be returned as change is a \$20 bill and pennies, nickels, dimes and quarters will be the only coins used. If a customer offers a \$20 bill for a purchase of \$12.18 then the change would be \$7.82. One \$5, two \$1's, three quarters, one nickel and two pennies.

Input: The first line in the input file will be a single number N which will not be greater than 20. N is the number of transactions. N lines of data will follow. Each line will contain the total amount of the purchase and the amount tendered by the customer separated by a single space. The amount tendered will always be greater than or equal to the amount of the purchase and no purchase will exceed \$100.00.

Output: The number of bills and coins in each denomination returned to the customer. The change for each transaction should be displayed all on one line from largest denomination to smallest. For each denomination show the denomination and the number of bills or coins. Use the following designations for each denomination: \$20, \$10, \$5, \$1, Quarters, Dimes, Nickels and Pennies.

If the number of coins is one, the denomination must be shown using the singular form i.e. Quarter, Dime, Nickel and Penny. If one or more of the denominations is not used in the change, it should not be displayed. If the amount tendered is exact change then print "No Change Owed".

Sample input:

```
6
12.18 20.00
10.00 20.00
5.50 5.50
0.53 1.00
3.59 10.00
13.59 50.00
```

Sample output:

```
1-$5 2-$1 3-Quarters 1-Nickel 2-Pennies
1-$10
No Change Owed
1-Quarter 2-Dimes 2-Pennies
1-$5 1-$1 1-Quarter 1-Dime 1-Nickel 1-Penny
1-$20 1-$10 1-$5 1-$1 1-Quarter 1-Dime 1-Nickel 1-Penny
```

9. Maite

Program Name: Maite.java

Input File: maite.dat

Maite is building a random number generator for a virtual board game she's developing. The video game is played with a deck of cards, where each card has a number on it. Because the players do not know either the contents or ordering of the deck, each game is quite dynamic.

When play-testing, Maite discovered players are unhappy when the same card appears too often. In an ideal game, no one card appears more than K times. To help implement this, Maite generates an array of N numbers, then chooses the largest subarray where no value appears more than K times. A subarray is a contiguous section within the original array. She's having issues implementing this last part. Can you help her?

Input: The first line is T ($T \leq 50$), the number of test cases to follow. The first line of each test case has two integers: N and K . The second line of each test case has N integers. The i -th integer is A_i .

$$\begin{aligned} 2 &\leq N \leq 200,000 \\ 1 &\leq K \leq N \\ -10^9 &\leq A_i \leq 10^9 \end{aligned}$$

The sum of N over all test cases does not exceed 2,000,000.

Output: For each test case, output the length of the largest subarray where no element appears more than K times.

Sample input:

```
3
6 2
1 3 1 3 1 3
7 3
2 7 7 2 7 7 2
9 1
1 4 7 10 3 6 9 2 5
```

Sample output:

```
Case #1: 4
Case #2: 5
Case #3: 9
```

Sample explanation:

The first array generated is [1, 3, 1, 3, 1, 3]. The longest subarrays where no value appears more than 2 times are [1, 3, 1, 3] and [3, 1, 3, 1] in middle and [1, 3, 1, 3] at end.

The second array generated is [2, 7, 7, 2, 7, 7, 2]. The longest subarrays where no value appears more than 3 times are [2, 7, 7, 2, 7] at front and [7, 2, 7, 7, 2] at end.

The third array generated has all unique elements, so we can take the whole array.

10. Mayank

Program Name: Mayank.java

Input File: mayank.dat

Mayank is fascinated with check digit algorithms and is really into cars. He knows that every car has a vehicle identification number or VIN and has discovered that VINs contain a check digit. A check digit is a character (sometimes a digit) added to a string of characters to help identify human keying errors. One such algorithm is used with VINs. Every on-road motorized vehicle like automobiles, trucks, and motorcycles are assigned a unique VIN. Since 1981, all U.S. on-road vehicles sold by dealers have a 17-character VIN. Groups of characters within the VIN identify the country and manufacturer, model year, information about the specific model like body style, engine, assembly plant code, and much more. Working from left-to-right, the ninth character is the check digit.

For example, in the VIN 1LNHM81W**Z**Y623687, the bold and underlined **Z** is the check digit and the following shows how the VIN check digit encoding works assuming that all other characters are correct.

Digits have their literal values while letters are assigned values as follows:

A: 1	B: 2	C: 3	D: 4	E: 5	F: 6	G: 7	H: 8	
J: 1	K: 2	L: 3	M: 4	N: 5		P: 7		R: 9
	S: 2	T: 3	U: 4	V: 5	W: 6	X: 7	Y: 8	Z: 9

Note that I, O, and Q do not have values because they are not used within VINs to avoid confusion with 1 and 0. The S seems out of position but the values are actually the low-order 4 bits of their EBCDIC codes (not ASCII).

Each position within a VIN is assigned a weight factor as follows:

Position	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Weight	8	7	6	5	4	3	2	10	0	9	8	7	6	5	4	3	2

The VIN character values and weights are multiplied together and the resulting products are added together:

VIN	1	L	N	H	M	8	1	W	7	2	Y	6	2	3	6	8	7
Value	1	3	5	8	4	8	1	6	7	2	8	6	2	3	6	8	7
Weight	8	7	6	5	4	3	2	10	0	9	8	7	6	5	4	3	2
Product	8	21	30	40	16	24	2	60	0	18	64	42	12	15	24	24	14

The sum of the products is 414 and the remainder when divided by 11 is 7 which is the check digit! When the remainder is 10, the check digit is X. When the check digit matches, the VIN is considered valid.

Mayank has asked your team for help writing a program to check the validity of VINs.

Input: First line contains the number of test cases, T, with $4 \leq T \leq 50$. Each of the next T lines contain a single VIN to be checked for accuracy. Each VIN contains only allowable characters with no leading or trailing spaces

Output: For each test case, display the test case number as shown below followed by a single space and either “VALID” or “INVALID”. When the VIN is invalid, display another single space and the calculated check digit.

Sample input:

```
4
1LNHM81W72Y623687
1LNHM81W27Y623687
1FTYR14C5WPA36284
1FTVR14C5WPA32648
```

Sample output:

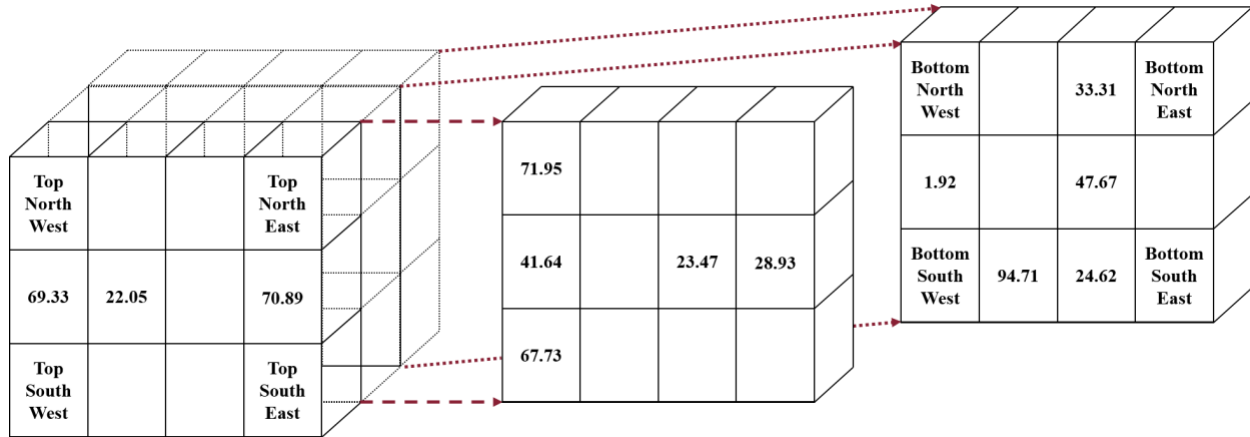
```
#1: VALID
#2: INVALID 8
#3: VALID
#4: INVALID 4
```

11. Rashi

Program Name: Rashi.java

Input File: rashi.dat

Rashi has been working to improve his programming skills and is currently struggling with a 3-dimensional array. He is viewing the array as a collection of layers where each layer contains rows of cells that are the columns. He envisions the layers progressing down from the sky towards the surface of the earth with the rows lined up north to south and the cells in the rows lined up west to east. It's as if you are looking down from the space station. The fronts of the following layers would actually be facing up towards the space station.



In the above diagram, the cells containing the position orientation all contain 0 data values as do the empty cells which are left blank so you can more easily visualize the meaningful data.

Rashi wants to find all of the clusters of data. A cluster is a collection of 2 or more cells with cell faces that touch one another. Obviously, the cells in the top layer containing 69.33 and 22.05 touch because they are side-by-side in the same row. The cells in the middle layer containing 71.95, 41.64, and 67.73 are side-by-side in a column. Because the 41.64 in the middle layer is immediately below the 69.33 in the top layer and immediately above the 1.92 in the bottom layer, those cells touch. That results in a cluster containing a total of 6 cells. The sum of their data values would be 274.62. The cell with 94.71 in the bottom layer touches 1.92 on a corner not a face and it touches 67.73 in the middle layer along an edge not a face so it is not included in the cluster. In a similar fashion, the cells containing 70.89, 28.93, 23.47, 33.31, 47.67, 24.62, and 94.71 are in a cluster with a sum of 323.60.

The second set of test data wraps the first set with a layer of 0s on all faces then adds full layers of data as a new top and bottom layer.

Input: First line contains the number test cases, $2 \leq T \leq 10$. Each test case starts with three positive integers each ≥ 2 and ≤ 100 , which are the number of layers (D for depth), rows (R), and columns (C) for the grid of data. The next $D \times R$ lines each contain C floating point values between 0.0 and 100.0, inclusive. The data rows start with the top layer of depth with rows from north to south and progressing to the deepest layer. The columns are from west to east – it's as if you were in the space station looking down at the earth.

Output: For each test case, display a line containing the test case # as shown below. Then display a sorted list of data cluster sums from largest to smallest with one cluster sum per line, rounded to two digits after the decimal point. Finish each test case with a row of 12 equal signs "===== ". When a test case has no clusters, display the message "NO CLUSTERS FOUND".

Sample input and output continued on next page ...

Rashi continued...

Sample input:

```
2
3, 3, 4
0, 0, 0, 0
69.33, 22.05, 0, 70.89
0, 0, 0, 0
71.95, 0, 0, 0
41.64, 0, 23.47, 28.93
67.73, 0, 0, 0
0, 0, 33.31, 0
1.92, 0, 47.67, 0
0, 94.71, 24.62, 0
7, 5, 6
70.26, 20.91, 45.92, 52.25, 54.55, 62.78
31.93, 73.24, 79.67, 7.65, 90.35, 13.28
59.07, 38.22, 64.00, 79.06, 76.54, 24.79
46.64, 8.04, 24.30, 18.86, 77.09, 98.87
70.94, 91.89, 80.49, 63.34, 72.44, 82.05
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 69.33, 22.05, 0, 70.89, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 71.95, 0, 0, 0, 0
0, 41.64, 0, 23.47, 28.93, 0
0, 67.73, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 33.31, 0, 0
0, 1.92, 0, 47.67, 0, 0
0, 0, 94.71, 24.62, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
70.50, 37.96, 5.14, 72.08, 30.61, 1.69
9.91, 25.47, 44.90, 14.91, 99.73, 19.65
46.76, 57.85, 2.34, 56.66, 71.04, 41.84
22.77, 84.80, 12.64, 67.51, 55.61, 74.38
21.87, 89.54, 8.02, 81.14, 3.71, 16.14
```

Sample output:

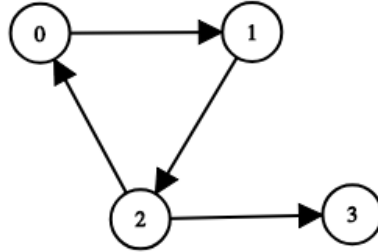
```
TEST CASE #1:
323.60
274.62
=====
TEST CASE #2:
1679.42
1247.17
323.60
274.62
=====
```


12. Sergio

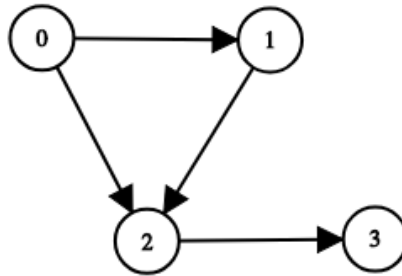
Program Name: Sergio.java

Input File: sergio.dat

In graph theory, a directed graph is a graph that is made up of a set of vertices connected by edges, where the edges have a direction associated with them. An example of a directed graph (Sample Input Test Case #1) is below:



In the above example, this directed graph contains a *cycle*. A *cycle* is defined as a non-empty trail in which the only repeated vertices are the first and last vertices. For example, if one were to start at Vertex 2, then $2 \rightarrow 0 \rightarrow 1 \rightarrow 2$ results in a trail in which the first and last vertex are repeated. The below directed graph (Sample Input Test Case #2) also contains a cycle. Notice that Vertex 1 has an edge to Vertex 1 (itself). So, a cycle exists since if one were to start at Vertex 1, then $1 \rightarrow 1$ results in a trail in which the first and last vertex are repeated



An example of a directed graph (Sample Input Test Case #3) without a cycle is shown below:



Notice, there does not exist a trail such that given a starting vertex, one could traverse back to the starting node.

Sergio would like you to write a program that given a directed graph, determines if the graph has a cycle or does not contain a cycle. Are you up to the challenge?

Input: Input starts with a line containing an integer N ($1 \leq N \leq 20$), the number of test cases. Each test case will begin with an integer V ($1 \leq V \leq 50$) on a single line, followed by another integer E ($0 \leq E \leq 50$) on a single line. V corresponds to the number of vertices the graph has, and E corresponds to the number of edges the graph has. The following E lines each have two integers in range from $[0, E]$ separated by a space. The first represents the source vertex and the second represents the destination vertex.

Output: For each test case output “Cycle” if the given graph contains a cycle, else output “No cycle” if the graph does not contain a cycle.

Sample input and output continued on next page ...

Sergio continued ...

Sample input:

```
7
4
4
0 1
1 2
2 0
2 3
2
2
0 1
1 1
4
4
0 1
0 2
1 2
2 3
6
6
0 1
1 2
2 3
3 4
4 5
5 0
6
5
0 1
1 2
2 3
3 4
4 5
5
5
0 1
0 2
1 2
2 0
2 3
5
4
0 1
0 2
1 2
2 3
```

Sample output:

```
Cycle
Cycle
No cycle
Cycle
No cycle
Cycle
No cycle
```